

系统工具开发课程实验报告（第四周）

廖世嘉 25020007073
25 级计算机科学与技术

2026 年 9 月 14 日

目录

1	实验基本信息	2
2	实验目的	2
3	实验环境与工具	2
4	实验十三：建立可执行的本地质量门禁	2
4.1	实验要求	2
4.2	配置文件	2
4.3	测试用例	3
4.4	质量检查脚本	3
4.5	结果	3
5	实验十四：让 Make 只重建真正受影响的产物	4
5.1	实验要求	4
5.2	源文件	4
5.3	Makefile	4
5.4	验证结果	5
6	实验十五：把本地 API 数据转换为可读报告	5
6.1	实验要求	5
6.2	数据文件	5
6.3	脚本实现	6
6.4	结果	6
7	实验十六：修复并交付一个陌生的小型工具仓库	7
7.1	实验要求	7
7.2	临时破坏与验证	7
7.3	修复	7
7.4	Makefile	7
7.5	构建与交付	7
8	实验结果汇总	8

目录	2
9 问题与解决方法	8
9.1 ruff 导入排序问题	8
9.2 Makefile 中的 Tab 缩进	8
9.3 jq 排序中的多字段排序	8
9.4 Git 提交的聚焦性	9
10 实验总结	9

1 实验基本信息

表 1: 实验基本信息

项目	内容
姓名	廖世嘉
学号	25020007073
专业	25 级计算机科学与技术
实验环境	Linux / Python / ruff / pytest / Make / curl / jq / Git
实验日期	2026 年 9 月 14 日

2 实验目的

本实验围绕代码质量门禁、Make 构建系统、API 数据处理与报告生成、以及综合仓库交付四个主题展开。通过四个连续实验，建立本地质量检查流程，掌握 Make 的增量构建机制，使用 curl 与 jq 处理 API 数据并生成 Markdown 报告，以及完成一次包含测试、构建、哈希计算和 Git 提交的综合检查。

3 实验环境与工具

本实验在 Linux 和 Windows 环境中完成，主要使用 ruff（代码格式化与检查）、pytest（测试框架）、Make（构建系统）、curl（HTTP 请求）、jq（JSON 处理）、python -m http.server（本地 HTTP 服务）、python -m build（wheel 构建）、sha256sum（哈希计算）以及 Git（版本控制）。实验目录为 SystemToolkitDevCourse，四个实验分别位于 q13、q14、q15 和 q16。

4 实验十三：建立可执行的本地质量门禁

4.1 实验要求

复制 q10 为 q13，在 pyproject.toml 中加入 ruff 和 pytest 的最小配置。补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则。编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest，运行并保证返回 0。

4.2 配置文件

pyproject.toml 中新增 ruff 和 pytest 配置：

```
[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
select = ["E", "F", "W", "I"]
```

```
[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["."]
```

4.3 测试用例

test_cli.py 包含两个测试：

```
import pytest
from greetlab.cli import main

def test_normal_name(capsys):
    import sys
    sys.argv = ["sdt-greet", "--name", "alice"]
    main()
    out = capsys.readouterr().out
    assert "Hello, alice!" in out

def test_blank_name_exits_nonzero():
    import sys
    sys.argv = ["sdt-greet", "--name", " "]
    with pytest.raises(SystemExit) as exc_info:
        main()
    assert exc_info.value.code == 2
```

4.4 质量检查脚本

check.sh 依次执行三项检查：

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== ruff format --check ==="
ruff format --check .

echo "=== ruff check ==="
ruff check .

echo "=== pytest ==="
python -m pytest test_cli.py -v

echo "=== All checks passed ==="
```

4.5 结果

运行 check.sh 输出：

```
=== ruff format --check ===
```

```
4 files already formatted
=== ruff check ===
All checks passed!
=== pytest ===
2 passed
=== All checks passed ===
```

退出码为 0，全部检查通过。期间 ruff 发现的导入排序问题已通过 `ruff check --fix` 修复，未使用全局忽略规则。

5 实验十四：让 Make 只重建真正受影响的产物

5.1 实验要求

在 q14 中创建 `data.csv`、`stats.py`、`build_report.py` 和 `report.md` 四个源文件，编写 Makefile 包含 `all`、`stats.txt`、`report.txt` 和 `clean` 目标。准确列出数据文件、脚本和中间产物依赖，将 `all` 和 `clean` 声明为.PHONY。验证首次 `make` 生成两个产物，第二次无改动时不执行配方，`touch data.csv` 后重新生成，`clean` 只删除生成物。

5.2 源文件

```
# data.csv
name,value
alpha,2
beta,3
gamma,5
```

`stats.py` 读取 `data.csv` 并计算总和写入 `stats.txt`：

```
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))
```

`build_report.py` 将 `report.md` 和 `stats.txt` 合并为 `report.txt`：

```
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

5.3 Makefile

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py
```

```
report.txt: report.md stats.txt build_report.py
python3 build_report.py

clean:
rm -f stats.txt report.txt
```

依赖链：data.csv 和 stats.py 生成 stats.txt，report.md、stats.txt 和 build_report.py 生成 report.txt。all 依赖 report.txt，clean 删除两个生成物。

5.4 验证结果

```
# 首次 make
$ make
python3 stats.py
python3 build_report.py

# 第二次 make（无改动）
$ make
make: Nothing to done for 'all'.

# touch data.csv 后
$ touch data.csv && make
python3 stats.py
python3 build_report.py

# clean
$ make clean
rm -f stats.txt report.txt
```

首次 make 生成两个产物，第二次 make 因无改动而不执行配方，touch data.csv 后 stats.txt 和 report.txt 均重新生成，clean 只删除生成物，保留源文件。Make 的增量构建机制工作正确。

6 实验十五：把本地 API 数据转换为可读报告

6.1 实验要求

在 q15 中创建给定的 packages.json，运行 python -m http.server 8000 启动本地 HTTP 服务。使用 curl -fsS 获取数据，使用 jq 筛选 status 为 active 且 downloads 不少于 100 的记录，按 downloads 降序、name 升序排列。编写 api_report.sh 将结果生成 summary.md，包含标题和 name/version/downloads 三列表格。

6.2 数据文件

packages.json 包含 5 条记录：

```
[
  {"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
  {"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
  {"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
```

```
{
  "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0",
  "name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0"
}
```

6.3 脚本实现

api_report.sh 使用 curl 获取 JSON 数据，通过 jq 进行筛选、排序和 Markdown 表格生成：

```
#!/usr/bin/env bash
set -euo pipefail

URL="http://127.0.0.1:8000/packages.json"
OUTPUT="summary.md"

curl -fsS "$URL" | \
jq -r '
  ["| name | version | downloads |", "|-----|-----|-----|"] as $header |
  [$header],
  ([.[] | select(.status == "active" and .downloads >= 100)]
   | sort_by(-.downloads, .name)
   | .[]
   | "| \(.name) | \(.version) | \(.downloads) |")
] | .[]' > "$OUTPUT"

{
  echo "# Package Summary"
  echo ""
  cat "$OUTPUT"
} > "${OUTPUT}.tmp" && mv "${OUTPUT}.tmp" "$OUTPUT"

echo "Report written to $OUTPUT"
```

6.4 结果

在 q15 目录启动 HTTP 服务后运行 api_report.sh，生成的 summary.md：

```
# Package Summary

| name | version | downloads |
|-----|-----|-----|
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

筛选结果正确：beta（inactive）和 epsilon（downloads=80 < 100）被排除。delta 和 gamma 的 downloads 均为 450，按 name 升序 delta 排在 gamma 之前。curl -fsS 在服务不可用时返回非零退出码，确保错误可被脚本捕获。

7 实验十六：修复并交付一个陌生的小型工具仓库

7.1 实验要求

复制 q13 为 q16 并初始化 Git。把问候语实现临时改成始终返回字面量 Hello, name!, 运行 check.sh 确认测试失败, 随后定位并修复。编写最小 Makefile 包含 check、build 和 clean, check 调用 check.sh, build 生成 wheel。运行 make check 和 make build, 计算 wheel 的 SHA-256, 并把最终改动提交为一个内容聚焦的 Git 提交。

7.2 临时破坏与验证

将 cli.py 中的实现临时改为字面量输出:

```
# 临时改坏
print("Hello, name!")
```

运行 check.sh 确认测试失败:

```
=== ruff check ===
F841 Local variable `a` is assigned to but never used
=== pytest ===
FAILED test_normal_name - assert "Hello, alice!" in "Hello, name!"
FAILED test_blank_name_exits_nonzero - Failed: DID NOT RAISE SystemExit
```

7.3 修复

将实现恢复为正确版本:

```
if not a.name.strip():
    raise SystemExit(2)
print(f"Hello, {a.name}!")
```

7.4 Makefile

```
.PHONY: check build clean

check:
    bash check.sh

build:
    python3 -m build --no-isolation

clean:
    rm -rf build dist src/*.egg-info .pytest_cache .ruff_cache
```

7.5 构建与交付

运行 make check 全部通过, 运行 make build 成功生成 wheel。计算 SHA-256:


```
$ sha256sum dist/*.whl
f5acb7abe8a6604d88a68cedc447857cf9b7add89e04c4a154ca489b630d1c6 dist/greetlab_25020007073
-0.1.0-py3-none-any.whl
```

Git 提交记录:

```
$ git log --oneline
778d461 fix: restore correct greeting and add Makefile with check/build/clean
c4cd828 init: greetlab package from q13
```

提交信息包含问题说明、修复内容和 SHA-256 哈希值，内容聚焦于本次修复和构建配置。

8 实验结果汇总

表 2: 四个实验的完成情况

题号	实验主题	验证结果
13	代码质量门禁	ruff format/check 和 pytest 全部通过, check.sh 返回 0
14	Make 构建系统	首次构建、无改动跳过、touch 重建、clean 清理均正确
15	API 数据转报告	curl+jq 筛选排序正确, summary.md 表格生成成功
16	综合测试	临时破坏验证失败, 修复后 make check/build 通过, SHA-256 已计算并提交

9 问题与解决方法

9.1 ruff 导入排序问题

在实验十三中, 初次运行 ruff check 发现 test_cli.py 的导入顺序不符合 isort 规则 (I001)。由于 pyproject.toml 中启用了 I 规则, ruff 要求导入按标准库、第三方库、本地模块的顺序排列。使用 ruff check -fix 自动修复后, 再次检查全部通过, 无需全局忽略规则。

9.2 Makefile 中的 Tab 缩进

在实验十四中, Makefile 的配方行必须使用 Tab 而非空格缩进。在 Windows 编辑器中保存时, 有时会将 Tab 转换为空格, 导致 make 报错 "missing separator"。解决方法是在 WSL 中使用 cat -A 检查 Makefile 中的缩进字符, 确保配方行以 开头。

9.3 jq 排序中的多字段排序

在实验十五中, 需要先按 downloads 降序、再按 name 升序排列。jq 的 sort_by 函数默认按升序排列, 对于降序需要在字段前加负号: sort_by(-.downloads, .name)。这一语法确保 delta 和 gamma 在相同 downloads (450) 时按 name 升序排列。

9.4 Git 提交的聚焦性

在实验十六中，初始提交误将临时文件 `commit_msg.txt` 一并提交。通过 `git rm --cached` 移除该文件并 `amend` 提交，确保最终提交仅包含 `.gitignore` 和 `Makefile` 两个必要文件的变更，符合“内容聚焦”的要求。

10 实验总结

通过本次实验，我建立了完整的代码质量保障流程：从 `ruff` 的格式化和静态检查，到 `pytest` 的自动化测试，再到 `check.sh` 的一键执行，形成了可重复的质量门禁。Make 构建系统的实验让我深入理解了文件依赖关系和增量构建的本质——只有当依赖比目标更新时才执行配方，这避免了不必要的重复构建。

实验十五让我掌握了 `curl` 与 `jq` 的组合使用方式：`curl -fsS` 确保请求失败时脚本中断，`jq` 的管道和 `select/sort_by` 函数可以高效地处理 JSON 数据。实验十六作为综合测试，将前序所有技能整合到一个可交付的 Git 仓库中，从临时破坏到修复、从构建到哈希计算、从测试到提交，形成了一次完整的“红——绿——交付”循环。

四周实验的核心收获是：好的工程实践不在于单个工具的使用，而在于将质量检查、构建系统、版本控制和自动化测试组合成一个可重复、可验证的工作流。`check.sh` 和 `Makefile` 就是这种工作流的物理载体——它们将隐性的操作步骤固化为显性的、可执行的命令序列。